



Product Lifecycle Visibility Platform

AI enabled application providing full visibility
into manufactured product lifecycle

The visibility gap: no unified view of the product lifecycle

Problem statement

The process from order placement through manufacturing, delivery, installation, and maintenance spans multiple systems and teams. Because these systems are fragmented, there is no unified view of the lifecycle of a customer order. Different teams rely on separate tools and manual coordination to understand the status of an order or product.

Three dimensions of the challenge

01

Fragmented systems of record

Lifecycle data lives across eight separate systems — CRM, ERP, PLM, MES, QMS, Logistics, Field Service, and Support. Each system serves one domain and one team. No system holds the full picture.

02

Manual coordination between teams

Without a unified view, teams rely on manual coordination to understand order status. This creates latency, leaves risks undetected and makes proactive communication to clients difficult.

03

No stakeholder has the right view

Clients need milestone and delivery visibility. Operations teams need progress, dependencies, and risk. Executives need delivery health and quality trends. None of these views exist today.

Problem Classification — and what the solution is to be

What this problem is NOT primarily about



Data quality or governance

The source systems contain the required data. It exists and is broadly accurate within each domain.



Process redesign

Manufacturing and operational processes function within each team's system. The problem is not how teams work in isolation.



Replacing source systems

The brief calls for a platform that overlays existing systems of record — not one that replaces CRM, ERP, MES, QMS, or any other system.

The core classification

A lifecycle visibility problem

No stakeholder can see where a given order stands across its full lifecycle. The data exists — but no platform assembles it into a coherent, unified view.

DATA FRAGMENTATION

Coherence gap across systems

Each source system sees one stage of the lifecycle. No system — and no person — can answer where any given order stands end-to-end without manually consulting multiple tools.

VISIBILITY ISSUE

Visibility gap across stakeholders

Clients, operations teams, and executive leadership each need a different view of the same lifecycle data but currently all three stakeholder groups lack end-to-end visibility.

Proposed Solution — A centralized lifecycle visibility platform that overlays existing systems of record and provides clear insights for clients, operations teams, and executive leadership across all seven lifecycle stages.

Siloed systems – no connecting thread

Each source system captures one slice of the lifecycle. Taken together they span the full journey — but no system, team, or person can see across all of them at once.

Source systems and what each owns

CRM

Order placement

Client and order information

ERP

Order management

Order fulfillment and financials

PLM

Product configuration

Bill of materials and engineering data

MES

Manufacturing progress

Production tracking and work orders

QMS

Quality assurance

Inspection results and compliance

Logistics

Shipping & delivery

Carrier tracking and logistics data

Field Service

Installation & setup

Field service and installation records

Support

Ongoing maintenance

Service requests and ticket history

The gap

None of these systems share a data model or communicate with each other. There is no shared order identifier across all systems, and no layer that stitches their outputs into a single lifecycle view. Status can only be reconstructed manually — and only partially.

What the new platform must do

The integrated solution is to serve as the connective layer — not a replacement for source systems, but the intelligence overlay that makes them coherent.

Core capabilities required

Pull data from all source systems

Integrate with CRM, ERP, PLM, MES, QMS, Logistics, Field Service, and Support — without replacing them.

Present a unified view of the lifecycle

Assemble fragmented data into a single coherent picture of where any order stands across all seven stages.

Provide transparency and operational intelligence

Go beyond status display — surface risks and issues stakeholders can act on before they compound.

Overlay existing systems, not replace them

The platform reads and interprets data from source systems. It does not own source data or replicate system functions.

What each stakeholder must be able to do

Clients

See order status, milestones, delivery timelines, and any issues affecting delivery or installation.

Operations teams

Track order progress, dependencies, delays, and risks across manufacturing, supply chain, logistics, and support.

Executive leadership

Monitor overall delivery health, manufacturing performance, quality trends, and potential risks to revenue or customer satisfaction.

Next: Solution architecture — how the solution is to fulfill these requirements: data integration, lifecycle intelligence, and stakeholder visibility.

Clients see their order as a journey

External-facing : Clients need clarity on where their order stands and confidence that they'll know if something changes — before they ask.

Order timeline			Order #ORD-2024-0418 · 3 units
	Order placement	Completed	Jan 8
	Product configuration	Completed	Jan 22
	Manufacturing	Completed	Mar 15
	Quality assurance	In progress	Apr 14
	Delivery	Pending	Apr 22
	Installation	Pending	May 3
	Service activation	Pending	TBD

Confidence indicators

Each upcoming stage shows an estimated date with a confidence signal — on track, watch, or at risk — derived from current progress and historical patterns.

Plain-language alerts

When an issue arises, clients see a translated, human-readable notice: "Manufacturing rework required — revised delivery estimate: June 3." No internal jargon.

Key documents

Technical specifications, certificates of conformance, and delivery confirmations are available to download as soon as they are generated — no email request required.

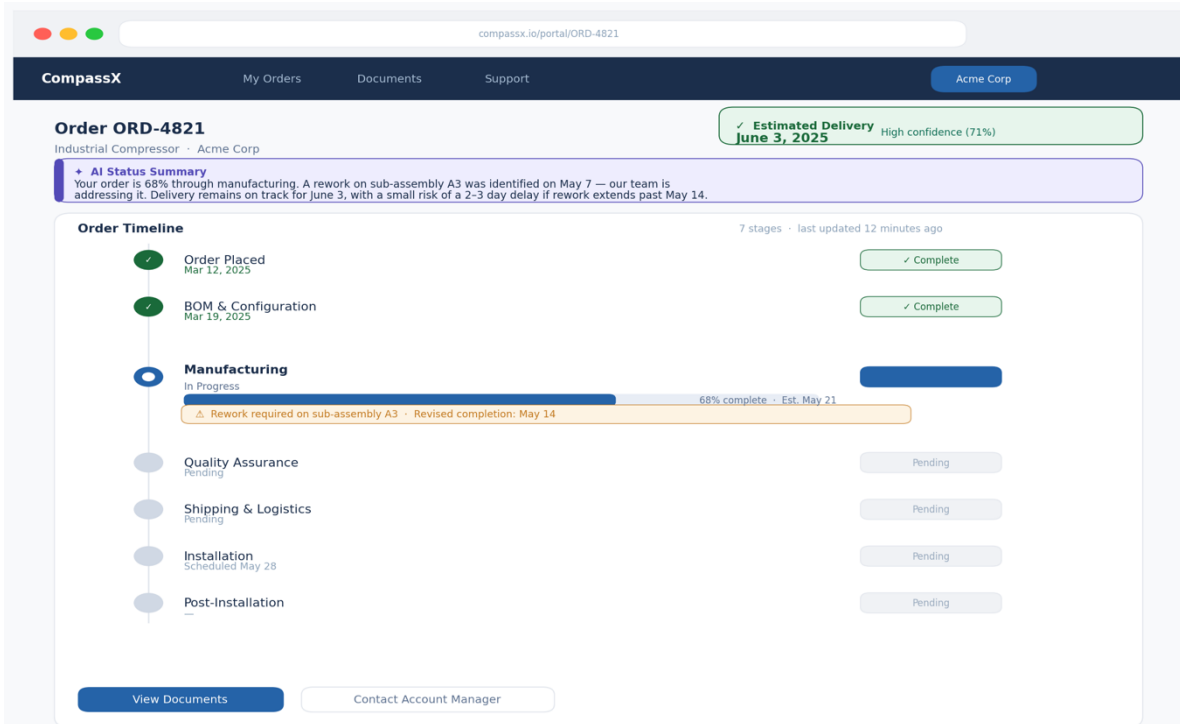
Single escalation point

Each alert surfaces a named contact and one-click escalation path, so clients never need to find the right person on their own when something goes wrong.

Design goal —

Clients feel informed and in control without needing to contact internal teams for routine status updates.

Clients Screen Mockup



Key design elements

AI status summary

Generated by claude-opus-4-6 on each view — translates lifecycle state into plain English the client understands.

Delivery confidence indicator

Driven by the XGBoost delay prediction model: p_on_time shown as Low / Medium / High confidence alongside the ETA.

Inline stage alert

Alerts appear directly on the affected stage — not in a separate panel — so clients immediately see what matters.

Progress bar per stage

In-progress stages show a % complete bar and estimated completion date, not just a status label.

The operations dashboard: portfolio, blockers, and alerts

Internal-facing: Multiple teams share one dashboard but filter to their own view. Primary question: what is blocked, and what is at risk?

Order portfolio view — filterable by stage, risk, customer, product line, and team

Order	Customer	Stage	Risk	Assigned team	Est. completion	Days late
ORD-0418	Acme Corp	Quality assurance	CRITICAL	Manufacturing	Apr 14 (+12d)	
ORD-0391	BetaTech	Manufacturing	AT RISK	Production	May 2 (+6d)	
ORD-0375	Nexivus	Delivery	ON TRACK	Logistics	Apr 18	
ORD-0362	Corplan	Installation	ON TRACK	Field Svc	Apr 25	

Bottleneck panel

Quality assurance		8	stalled
Manufacturing		5	stalled
Delivery		3	stalled

Dependency view

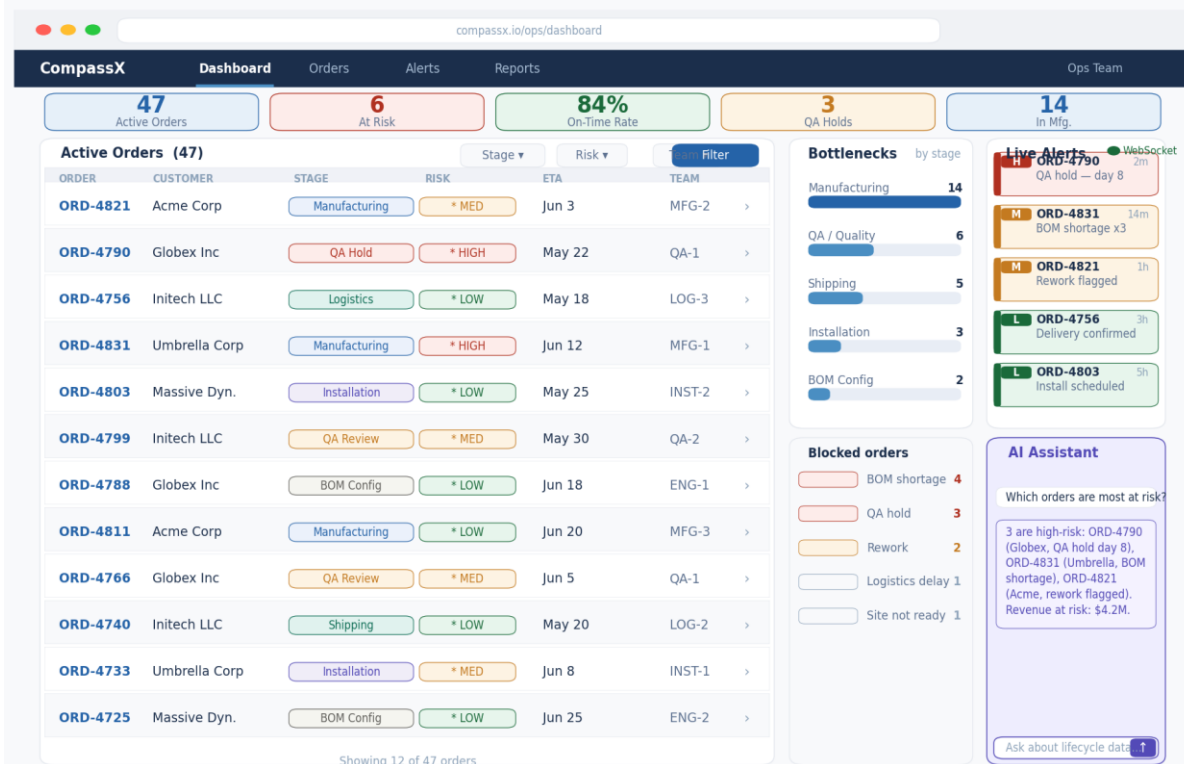
BOM shortage (PLM)	4 orders blocked
Quality hold (QMS)	2 orders blocked
Logistics delay	3 orders affected

Live alert feed

CRITICAL	ORD-0418 — QA hold: rework on unit 2/3. Delivery impacted.	2 min
WARNING	ORD-0391 — BOM shortage: 2 parts on backorder. +6 days.	18 min
INFO	ORD-0375 — Shipment confirmed. Delivery: Apr 18, 14:00.	1 hr
INFO	ORD-0362 — Install scheduled. Field team: J. Torres.	3 hr

Design goal — Each team gets what needs their attention — no status meeting required to find where orders stand or what is blocking them.

Operations Screen Mockup



Key design elements

Portfolio table

47 active orders with inline risk badges and stage pills. Filter by stage, risk level, or team. One click to order detail.

Live alert feed

WebSocket-powered. Alerts appear in real time without page refresh. Color-coded by severity with one-click drill-down.

Bottleneck panel

Stage-by-stage bar chart showing where orders are stacking. Blocked/stalled breakdown by root cause below.

AI Assistant

Embedded in the dashboard. Routes aggregate queries to DB, specific orders to exact match, exploratory questions to RAG.

Executives see a high-level command center

Internal-facing: Executives need portfolio-level health, trends, and early warning of risks that could affect customers or revenue. The dashboard answers: are we performing, and what should I be concerned about?

On-time delivery

73%

↓ 4pts vs last quarter

Avg. cycle time

68d

↑ 5 days vs target

Quality pass rate

91%

Stable — within range

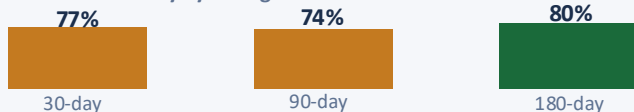
Orders at risk

6

2 critical, 4 warning

Delivery performance trends

On-time delivery by rolling window



Revenue at risk

Order	Value
● ORD-0418	\$1.2M
● ORD-0391	\$840K
● ORD-0388	\$610K

AI-generated risk narrative

Daily summary · Apr 8

6 orders are at elevated risk. The largest exposure is ORD-0418 (\$1.2M), currently in QA hold with a revised delivery of Jun 3. Manufacturing throughput is running 11% below target — if not recovered by Apr 15, 3 further orders will breach SLA.

What the dashboard enables

Spot systemic issues

Quality pass rate dips or cycle time growth visible across rolling windows before they compound into customer complaints.

Track revenue exposure

Delayed orders linked directly to contract value — not just a list of late orders but a ranked view of financial exposure.

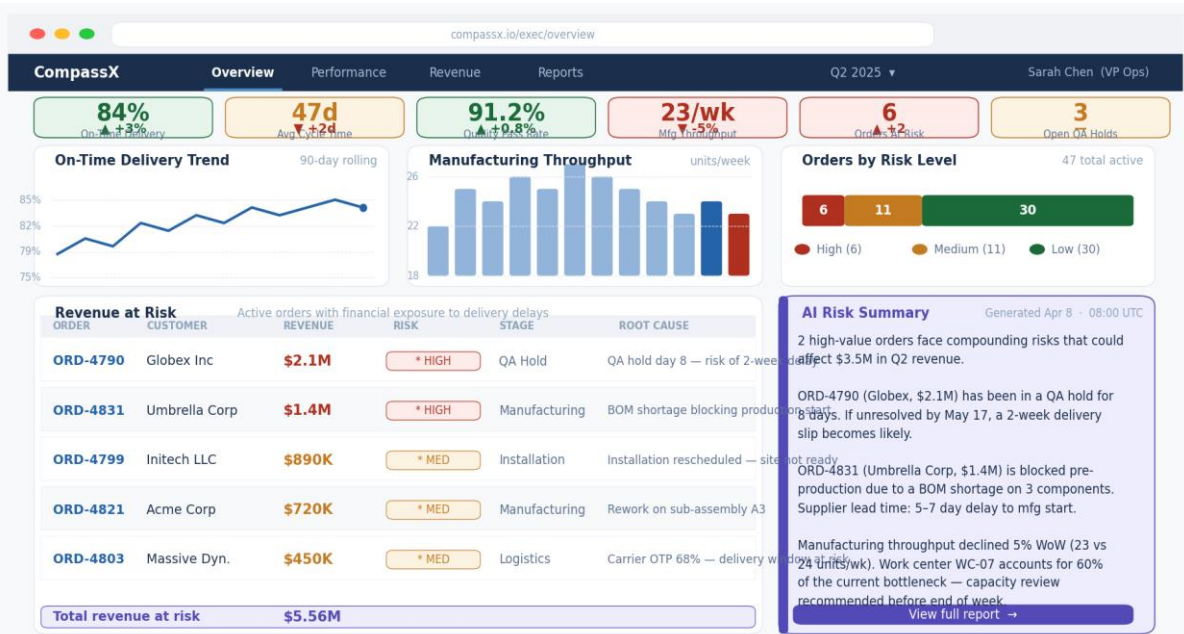
Intervene early

The narrative flags the highest-priority risks in plain language, so leadership can act before issues become customer-impacting events.

Design goal —

Leadership can spot systemic issues, track performance trends, and intervene before risks become customer-impacting events.

Executives Screen Mockup



Key design elements

KPI cards with trend

Six KPIs with period comparison and directional arrows: OTD %, cycle time, quality pass rate, throughput, orders at risk, QA holds.

Revenue at risk table

Operational issues linked directly to financial exposure. Each row shows order, customer, revenue amount, root cause, and risk tier.

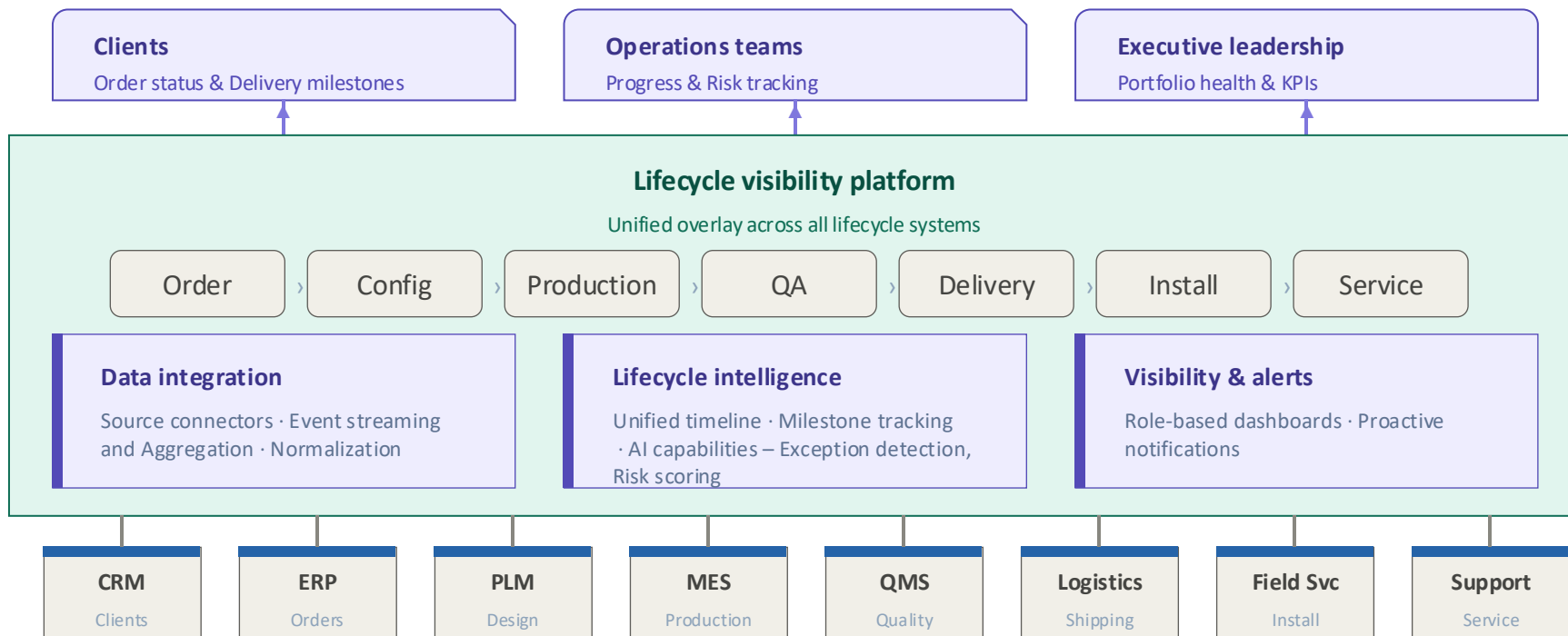
Trend charts

OTD sparkline (90-day rolling), throughput bar chart with WoW comparison, and orders-by-risk stacked bar.

AI daily narrative

Generated at 07:00 UTC via EventBridge. Concrete, actionable language — not a summary of metrics, but an interpretation of risk.

The Lifecycle Visibility Platform – At a glance

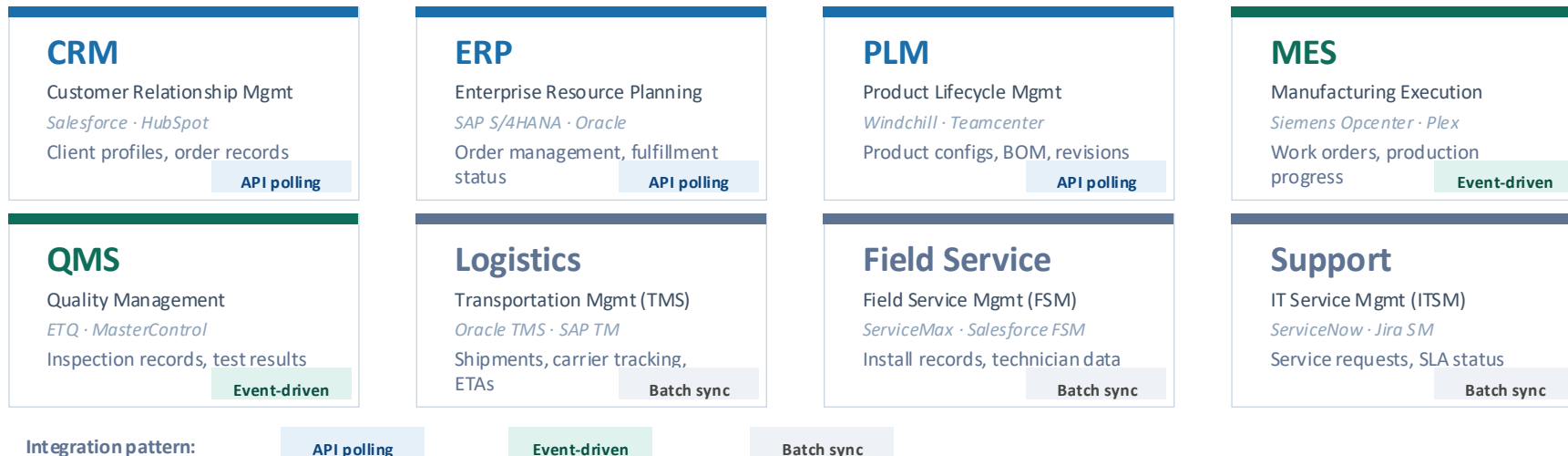


Design principle —

The platform overlays existing systems. It reads and normalizes their data – no source system is replaced or disrupted.

Eight source systems – and data they contribute

The platform connects to all eight system categories via the adapter pattern — mock connectors in the initial build, each swappable for a live connector without changing platform logic.



Adapter pattern — Each connector is built to a common interface. Swap any mock for a live connector independently — no changes to platform logic required.

The Order as the lifecycle container

Every entity links back to Order via `order_id`. Status at each stage is computed from events – views are derived from source data.

Entity hierarchy

Order

Primary anchor · all entities link via `order_id`

ProductConfiguration

one per order

BillofMaterials

line items

ProductionWorkOrder

one or many

QualityInspectionRecord

per work order

Shipment

one per order

DeliveryEvent

milestones

InstallationRecord

one per order

ServiceTicket

recurring

What every entity carries

source_system

Which integration provided this record

external_id

The record ID in the originating source system

last_synced_at

Timestamp of the most recent pull from that system

data_confidence

fresh / stale / missing — from sync age thresholds

source_system + external_id provide a complete trace back to the originating record in any source system.

Key design rule — Status is computed from events, never stored as mutable state.

Handling fragmented and inconsistent data

Challenge

Missing stage data

No event has arrived from a source system for one or more stages.

Conflicting records

The same field arrives from two systems with different values.

Stale data

A connector goes silent while outdated data continues to display as current.

Schema variations

Each system uses its own field names, ID formats, date formats, and enumerations.

Duplicate events

Retries, webhook replays, or overlapping polls push the same event more than once.

How the platform handles it

Stage status defaults to 'unknown' — never fabricated. The `data_confidence` field (fresh / stale / missing) drives a visible indicator. Display is never blocked; gaps render explicitly.

Reconciliation priority resolves conflicts per entity type — ERP overrides CRM for order status; QMS is authoritative for quality results. Priority rules are configurable per field.

When `last_synced_at` exceeds a per-connector threshold, a staleness indicator surfaces in the UI alongside the data. Each card shows the age of its source data explicitly.

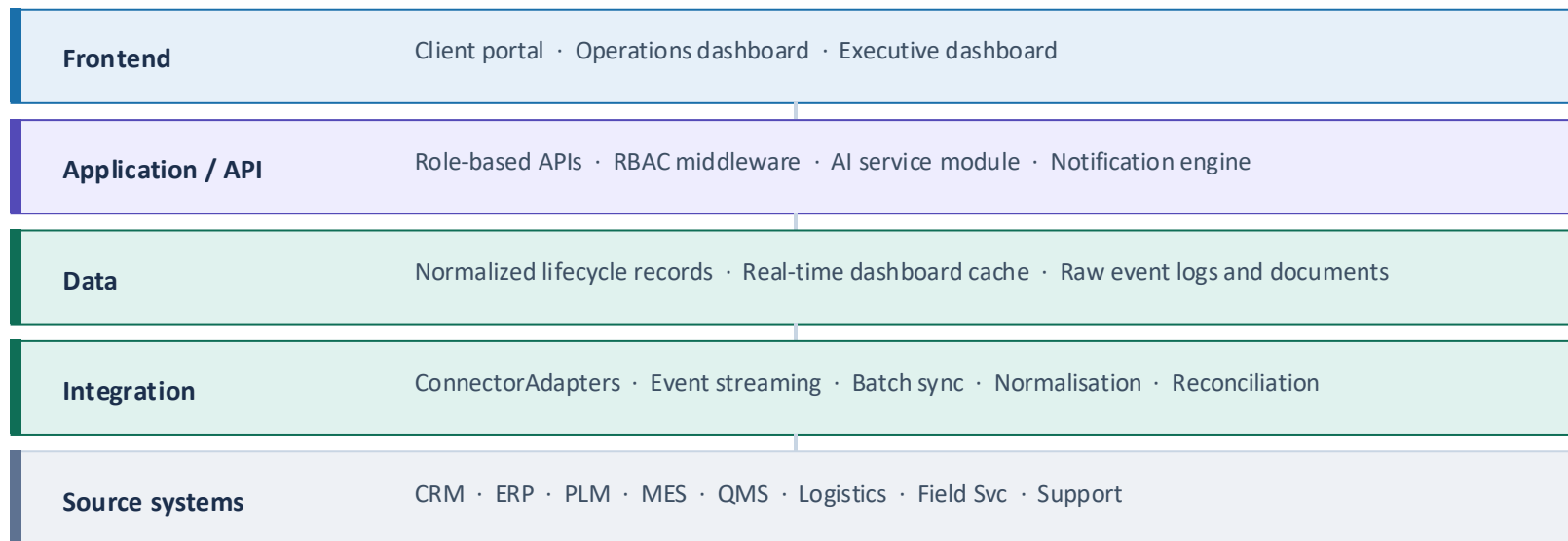
Each `ConnectorAdapter` translates source data into the canonical `CompassX` entity model before storage. Schema translation is isolated per connector — internal model never changes.

Ingestion workers are idempotent: records upsert by `(source_system, external_id)`. Reprocessing the same event any number of times produces no duplicate rows or state changes.

Design principle — Unknown is explicit, stale is labelled, every conflict has a defined resolution rule.

Application layers: from source to views

The platform is structured as five layers, read bottom-up: source data flows up through integration and storage into the application tier, which serves each audience the view they need.



Security & Access Control is a cross-cutting concern applied at every layer — Authentication, Role-based Data scoping, and Audit logging applies to all tiers.

Technology stack — layer by layer

Python-first, cloud-native on AWS. Each layer deploys independently as a container. Mock connectors in the initial build swap for live integrations without changing any downstream logic.

Application layers

Frontend	React + TypeScript · React Query Role-based routing via JWT claims
Application / API	FastAPI · WebSocket · RBAC AI module: Summarize · Risk score · Predict delay
Data	PostgreSQL / RDS — lifecycle records Redis / ElastiCache — dashboard cache Amazon S3 — event logs, documents
Integration	SQS · EventBridge · ConnectorAdapters Workers → Normalizer → Reconciler
Source systems	CRM · ERP · PLM · MES · QMS Logistics (TMS) · Field Service · Support

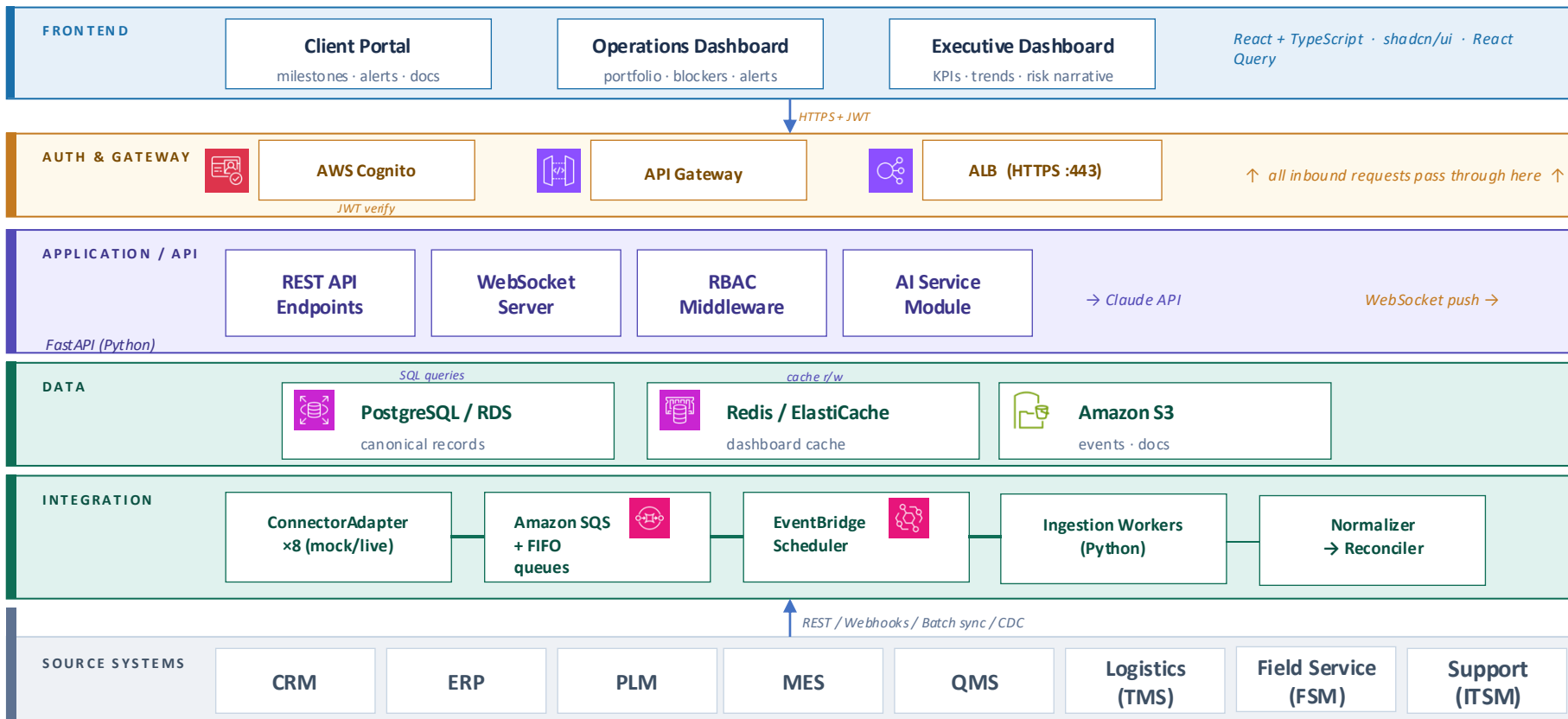
Cross-cutting concerns

Authentication	AWS Cognito · OAuth 2.0 / OIDC · JWT JWT claims carry role for RBAC routing
API gateway	AWS API Gateway · rate limiting · routing SSL termination before FastAPI services
Infrastructure	Terraform IaC — all AWS resources Version-controlled · reproducible
CI / CD	GitHub Actions → ECR → ECS Fargate Per-service deploys · auto rollback
Observability	CloudWatch) · Per-service metrics · Alert Thresholds

Deployment model —

All services containerized on AWS ECS Fargate. Infrastructure reproduced via Terraform. No manual provisioning.

Layer-wise Components



AWS deployment topology — VPC, compute, and data services

AWS Account — us-east-1 (Multi-AZ: us-east-1a / us-east-1b)

PUBLIC SUBNETS



Route 53

① Users → CloudFront → S3 SPA (static)



CloudFront

+ S3 SPA



ALB (HTTPS :443)



API Gateway

webhook alt

NAT Gateways

per AZ

PRIVATE SUBNETS

② API calls → ALB → api-service (:8000)

④ Worker polls SQS → upserts PostgreSQL, invalidates Redis

③ Webhooks → ALB or API Gateway → SQS

ECS FARGATE CLUSTER



api-service

FastAPI · 1 vCPU · 2 GB
2–8 tasks · ALB autoscale
REST + WebSocket + AI

worker-service

Ingestion Workers · 0.5 vCPU · 1 GB
1–4 tasks · SQS autoscale
normalize · reconcile · upsertPostgreSQL
RDS Multi-AZ
+ RDS Proxy
+ pgvectorElastiCache
Redis cluster
2 shards
cache + pub/subSQS FIFO
8 queuesEvent-
Bridge
Scheduler

S3 BUCKETS



compassx-events

Raw Payloads
(Glacier 90d)

compassx-documents

Client Docs
(no expiry)

compassx-ml-models

XGBoost
(Versioned)

compassx-exports

CSV/PDF
(Expire 7d)

AWS MANAGED



Cognito



Secrets Mgr

CloudWatch

ECR

SOURCE SYSTEMS

CRM · ERP · PLM · MES · QMS · Logistics · Field Service · Support

Claude API (Anthropic)

external HTTPS · AI service module calls

Internet (users)

HTTPS → CloudFront or ALB

ECS Fargate services

api-service

Runtime	FastAPI · uvicorn (single worker per container)
Resources	1 vCPU · 2 GB RAM per task
Scaling	Desired: 2 tasks (HA) · autoscale 2–8 on ALB request count
Network	Port 8000 · ALB target group · health: GET /health
Endpoints	REST /api/* · WebSocket /api/ws/alerts · POST /webhooks/*
AI module	summarize_order() · score_risk() · predict_delay() · ask()
Secrets	DB URL, Redis URL, Cognito pool ID, Claude API key — injected from Secrets Manager
Deploy	Rolling update via ECS · ALB drains connections before termination

worker-service

Runtime	Python · async SQS polling loop (all 8 queues round-robin)
Resources	0.5 vCPU · 1 GB RAM per task
Scaling	1–4 tasks · autoscale: SQS approx. message count (< 500 / task)
Network	No inbound · outbound only: RDS, Redis, S3, source system APIs
Scheduler	EventBridge Scheduler: one rule per connector · 1–15 min interval
Pipeline	Dequeue → normalize → reconcile → invalidate cache → alert → ack
IAM	Least-privilege task role: SQS r/w, S3 put, Secrets r, RDS IAM auth
Deploy	Same CI/CD pipeline as api-service · independent image + task def

Shared IAM task role — Both services use RDS IAM database authentication — no static passwords in environment variables or images.

Integration and Ingestion

Two ingestion paths

Polling (REST-based: CRM, ERP, PLM)

- ① EventBridge Scheduler fires rule (per-connector, 1–15 min)
- ② Triggers ECS Run Task · worker calls `fetch_updates`
- ③ Raw events written to S3 `compassx-events/{source}/{date}/`
- ④ Events published to source's SQS FIFO queue
- ⑤ `last_run_timestamp` updated in `connector_sync_state` table

Webhook (near-real-time: MES, TMS)

- ① Source POSTs to `/webhooks/{source}` on ALB
- ② FastAPI validates HMAC signature (per-connector secret in Secrets Manager)
- ③ Payload enqueued to SQS immediately · handler returns 200 in < 5 ms
- ④ SQS message processed asynchronously by worker-service

Common worker processing (after dequeue)

Deserialize → `DataNormalizer.normalize()` → `Reconciler.upsert()`
 Invalidate Redis keys → emit alert if threshold crossed
 Write raw payload to S3 → delete SQS message (ack)

SQS FIFO queue design

Queue design

8 FIFO queues — one per source: `compassx-crm.fifo` ... `compassx-itsm.fifo`
`MessageGroupId` = `order_id` → ordered per-order across concurrent workers

Reliability

Visibility timeout 60 s · max receive count 3 · DLQ: `compassx-dlq.fifo`
`DLQ message count > 0` → CloudWatch alarm → SNS on-call notification

Idempotency

UPSERT by (`source_system`, `external_id`) — safe to reprocess any message
 Duplicate events produce no duplicate rows or phantom state changes

Adapter contract

Abstract base: `fetch_updates(since)` and `handle_webhook(payload)`
 Mock adapters: Faker data seeded by `order_id` · real adapters: identical signatures

FIFO guarantee — `MessageGroupId` = `order_id` ensures events for the same order are processed in arrival order, even with concurrent workers.

Data layer: PostgreSQL, Redis, and S3

PostgreSQL / RDS

Instance

db.t3.large · Multi-AZ · RDS Proxy
(connection pooling)

Read replica

Executive dashboard queries (KPIs, trends)
routed here

pgvector

lifecycle_embeddings table (1536-dim) for
RAG — avoids separate vector DB

Key tables

- orders · production_work_orders
- quality_inspections · shipments
- delivery_events · installation_records
- source_system · external_id
- last_synced_at · org_id · service_tickets

Tenant isolation

ORM base class: WHERE org_id = :org_id on
every query

Auth

IAM database authentication · no static
password

Redis / ElastiCache

cache.r7g.large · cluster mode · 2 shards

Key pattern	Content	TTL
<code>orders:portfolio:{org}:{role}:{hash}</code>	Paginated order list	60 s
<code>orders:{order_id}:lifecycle</code>	Full lifecycle detail	30 s
<code>kpis:{org}:executive:{period}</code>	KPI aggregates	5 min
<code>alerts:{org}:recent</code>	Last 50 unread alerts	10 s

Pub/Sub (separate from cache)

Channel alerts:{org_id} — Reconciler publishes on risk
threshold cross · all api-service tasks subscribe · each task
pushes to connected WebSocket clients for that org

Cache invalidation

Worker explicitly deletes orders:{order_id}:lifecycle and
portfolio keys after each UPSERT · KPI keys expire on TTL

*WebSocket heartbeat: ping/pong every 30 s · JWT validated on
connect via ?token= param*

Amazon S3

compassx-events

Raw event payloads
audit trail · reprocessing

Glacier after 90 d

compassx-documents

Client-facing documents
specs · certs · confirmations

No expiry

compassx-ml-models

Serialised XGBoost files
delay prediction + risk scoring

Versioned

compassx-exports

CSV / PDF report exports
generated on demand

Expire 7 d

pgvector — Lifecycle summaries are embedded and stored in PostgreSQL. The ask() endpoint uses top-5 RAG retrieval — no separate vector DB required.

Application layer, authentication, and frontend

FastAPI module structure

Routers

orders.py · manufacturing.py · logistics.py
alerts.py (REST + WS /api/ws/alerts) · webhooks.py · ai.py

Services

lifecycle.py · ai_service.py · alert_service.py

Middleware & models

auth.py (JWT + RBAC) · tenant.py (org_id scoping)
SQLAlchemy ORM models · Pydantic schemas

Authentication & tenant isolation (Cognito + JWT)

- ① PKCE flow from React → Cognito issues RS256 JWT (1 h) + refresh token (30 d, httpOnly cookie)
- ② Access token stored in memory only — never localStorage
- ③ FastAPI verify_token validates JWT against Cognito JWKS endpoint
- ④ Extracts custom:role + custom:org_id → attached to request context
- ⑤ Route decorators enforce role: @require_role('ops') / @require_role('executive')
- ⑥ ORM base class injects WHERE org_id = :org_id on every query (tenant isolation)
- ⑦ SAML 2.0 federation supported via Okta or Azure AD

Frontend — React SPA

Build & delivery

Vite build → dist/ → GitHub Actions → S3 compassx-frontend
CloudFront invalidation after S3 sync · ~30 s propagation
Single origin for both SPA and API — no CORS
React Router v6 · role-based routing from JWT claims

React app & state management

React Query: all server state — order lists, lifecycle, KPIs, alerts
Mutations: alert acknowledgment with optimistic updates
WebSocket: receives live alert pushes without polling
Vite dev proxy: /api → localhost:8000 mirrors CloudFront

AI service module

summarize_order(id, audience) → claude-opus-4-6 · cached 10 min
score_risk(id) → XGBoost + claude-haiku 1-sentence explanation
predict_delay(id) → probability + days at risk (XGBoost)
ask(question, org_id) → pgvector top-5 RAG + claude-opus-4-6

Cognito federation — Supports SAML 2.0 via Okta or Azure AD for enterprise SSO. 3 user pool groups: clients, ops, executives. Custom attributes: org_id, role.

CI/CD pipelines, observability, and secrets management

Automated pipelines (GitHub Actions, triggered on merge to main)

Backend (api-service + worker-service)

- ① pytest (unit + integration tests against local PostgreSQL container)
- ② ruff (linter) + mypy (static type checker)
- ③ docker build → docker tag compassx-api:{git-sha}
- ④ Push image to Amazon ECR
- ⑤ Register new ECS task definition revision (updated image URI)
- ⑥ ECS rolling update: tasks replaced one at a time · ALB drains connections
- ⑦ Steps ③–⑥ repeated independently for worker-service image

Frontend (React SPA)

- ① vitest (unit tests) + eslint + tsc --noEmit
- ② vite build (production) → dist/
- ③ aws s3 sync dist/ s3://compassx-frontend --delete
- ④ aws cloudfront create-invalidation --paths '/*'

Infrastructure (Terraform)

terraform plan on every PR · output posted as PR comment
 terraform apply requires manual approval · state in S3 + DynamoDB

Structured logging, metrics, alarms, and secrets

Logging & metrics

structlog: structured JSON · every entry includes service, trace_id, org_id, order_id
 CloudWatch Logs: /ecs/api-service + /ecs/worker-service + AWS X-Ray tracing
 Custom metrics: IngestLag · QueueDepth · CacheHitRate · AIResponseLatency

CloudWatch alarms → SNS → on-call

SQS DLQ message count > 0 (ingestion failure)
 API 5xx error rate > 1% over 5-minute window
 RDS connection count > 80% of RDS Proxy limit
 Worker Ingest Lag > 5 minutes (stale data risk)

Secrets management (AWS Secrets Manager)

Credentials stored by ARN: DB password, Redis auth, Claude API key, Cognito client secret, per-connector HMAC secrets
 ECS task definitions reference secrets by ARN · injected as env vars at startup
 Never baked into container images · rotated without redeployment

End-to-end latency target — Dashboard display in < 2 seconds for webhook-based. Reflect within 1–15 minutes for Polling-based sources .

AI Capabilities: From Reactive reporting to Proactive intelligence

AI is embedded at multiple points in the platform. Each capability uses the approach best suited to the problem – supervised ML where labeled history exists, statistical methods where interpretability matters, and LLMs where natural language output is required.

1 Delay Prediction

Supervised ML — XGBoost gradient boosting
Structured tabular features from lifecycle DB
Runs on every ingestion event as fast inference

Haiku — 1-sentence explanation only

2 Anomaly Detection

Statistical process control (SPC) – monitoring time series
CUSUM for sustained drift + Z-score for spikes
No training data required – numpy only

Haiku — plain-English alert description

3 Lifecycle Summaries

Structured prompting — not free-form generation
Audience-specific system prompts

claude-opus-4-6 — primary response model

4 Risk Scoring

Composite weighted score 0–100
ML output + four rule-based components
Decomposed into contributing factors

Haiku — 1-sentence score explanation

5 AI Assistant (RAG)

Retrieval-augmented generation
pgvector semantic search + query routing
Grounded entirely in live lifecycle DB data

claude-opus-4-6 — response generation

Claude API usage by capability

Capability	Model	Trigger	Volume
Lifecycle summaries	claude-opus-4-6	Cache miss + daily pre-gen at 07:00 UTC	Low · cached 10 min
AI assistant	claude-opus-4-6	On each user question	Low–medium · interactive
Risk score explanation	claude-haiku-4-5-20251001	After every ingestion event	High · automated, frequent
Anomaly description	claude-haiku-4-5-20251001	When anomaly is detected	Low · event-driven

Cost management — Cache summaries aggressively · Route aggregate questions away from LLM · Use Haiku for all high-volume automated calls

Predict on-time delivery probability for every active order

Supervised ML using XGBoost gradient boosting. Input is structured tabular data; training labels (actual vs. promised delivery date) come from historical order records. Inference runs automatically on every ingestion event — not just on demand.

Features — point-in-time snapshot from lifecycle DB

Feature	Note
Order age (days)	Elapsed since order placement
BOM completion %	Confirmed items / total BOM items
Open BOM shortage flags	Count of unresolved supply
Manufacturing progress %	Work orders completed / total
Days in current stage	Stall indicator
QA pass rate (90-day rolling)	For this order's product family
Count of QA holds	Open holds on this specific order
Carrier OTP % (90-day rolling)	From TMS history for assigned
Avg cycle time	Historical for this product category
Customer region	Installation distance proxy

Model training

Algorithm:	XGBoost
Label:	days_late = actual_delivery_date – promised_delivery_date
Training:	All completed historical orders (ERP + lifecycle DB)
Split:	80% train / 20% test · evaluation metric: RMSE
Cadence:	Retrained monthly via EventBridge-scheduled ECS task
Promotion:	New model replaces live only when RMSE improves

Inference pipeline — triggered after each DB upsert for an active order

1	Ingestion worker completes DB upsert for active order
2	10-feature vector assembled from current DB state (Python)
3	XGBoost model loaded from S3 — cached in memory on worker startup
4	Inference: p_on_time + predicted_delay_days computed in < 1 ms
5	Results written to order_predictions table with timestamp
6	Values consumed by risk scorer and client portal confidence indicator

Cold start — Initial build uses 6 months of synthetic historical orders, replaced with real-data model as production history accumulates.

Detect manufacturing deviations before they impact specific orders

Statistical process control (SPC) – Signals have well-defined historical baselines; SPC is fully interpretable, requires no training data, and responds within a single observation window. Implemented in – no external ML library.

Two detection methods

CUSUM — Cumulative Sum Control Chart

Detects sustained directional drift across multiple observations

Example: work center throughput declining for 5 consecutive shifts

Sensitivity $k = 0.5\sigma$ · alert threshold $h = 5\sigma$ accumulation

Runs after each batch of daily metrics is ingested

Z-score — Single-observation spike detection

Detects single-observation outliers against a rolling historical baseline

Example: QA pass rate drops from 94% to 61% in one day

Alert fires when $|z| > 3$ (> 3 standard deviations from rolling mean)

Metrics monitored (per work center / product family / supplier)

QA pass rate	MES throughput	BOM delivery rate	Avg work order time
daily	per shift	per supplier, weekly	7-day rolling

AnomalyRecord — written to PostgreSQL on each detection

anomaly_type	CUSUM_DRIFT Z_SCORE_SPIKE
metric	qa_pass_rate throughput bom_delivery_rate ...
entity_type	work_center product_family supplier
deviation_magnitude	z-score or CUSUM accumulation statistic
baseline_value	Rolling historical mean for this entity
observed_value	Value that triggered the alert
affected_order_ids	All orders at this work center / using this supplier

How anomalies surface in the platform

Operations Dashboard alert feed — plain-English via claude-haiku-4-5-20251001:

"Work center WC-07 throughput has declined 18% over 6 shifts (CUSUM alert).

4 active orders are assigned to this work center."

Active anomaly → risk score boost for all affected orders (anomaly_component in risk scorer, Section 5.4)

Why SPC —

Signals have well-defined baselines, SPC fully interpretable to operations teams, requires zero labeled training data

Plain-language order status for every audience, generated on demand

Structured prompting via claude-opus-4-6. Tightly controlled by a prompt that provides the full current lifecycle state as structured context and constrains output length and register. Audience-specific system prompts produce different outputs from the same lifecycle data.

Context assembly (before every Claude API call)

ai_service.summarize_order() fetches and assembles:

```
{ "order": { "id": "ORD-4821", "product": "Industrial Compressor",
  "customer": "Acme Corp", "promised_date": "2025-06-03" },
  "current_stage": "manufacturing",
  "stage_statuses": [ 7 stages, each with status + timestamps ],
  "delay_prediction": { "p_on_time": 0.71, "predicted_delay_days": 3 },
  "risk_score": 42,
  "open_alerts": [ { "type": "manufacturing_hold", ... } ] }
```

Caching and freshness

Redis TTL: 10 minutes per order per audience type

Cache invalidated when ingestion worker writes a new event for that order_id

Cache miss: regenerated synchronously (Claude API latency: 1–3 s)

Executive digest: all at-risk summaries pre-generated at 07:00 UTC via EventBridge

Stored in order_summaries with audience, generated_at, model_version for audit

System prompt — client audience

You are CompassX, an order status assistant. Write a 2–3 sentence update for a client about their order. Use plain language — no internal codes, system names, or manufacturing jargon. Focus on what the client cares about: where their order is, when it will arrive, and whether there are any issues they should know about. If there is a delay risk, state it clearly and give a revised estimate. Do not speculate beyond the data provided.

System prompt — executive audience

You are CompassX, an operational intelligence assistant. Write a 2–3 sentence executive briefing on this order. Include the current stage, any active risks with business impact, the delay probability, and the key operational factor driving any risk. Use precise operational language. Reference specific metrics where available.

How it works

The same lifecycle JSON produces two distinct outputs — the LLM formats and interprets, never invents. Summaries are constrained to what is in the structured context block.

Grounded generation — The prompt explicitly instructs, Output controlled, audited, and tied to the structured context — not free-form generation.

Risk score per order – decomposed into contributing factors

Composite score combining the ML delay prediction output with four rule-based signals. Rules are explicit and auditable; the ML component adds predictive power beyond what rules alone can capture. Runs after every ingestion event for every active order.

Score computation — weighted sum clamped to [0, 100]

Component	Weight	Logic / source
delay_prediction	0.35	1 - p_on_time from XGBoost model
bom_risk	0.20	shortage_count / total_bom_items + unconfirmed_pct × 0.5
quality_risk	0.20	open_qa_holds / total_inspections + 0.3 if rework_required
anomaly	0.15	0.0 (none) → 0.5 (stage anomaly) → 1.0 (critical)
logistics	0.10	0.0 (carrier OTP > 95%) → 1.0 (OTP < 70% or delayed)

Risk tiers

0–30 LOW On track	31–60 MEDIUM Monitor closely	61–100 HIGH Intervention required
---------------------------------------	--	---

Score decomposition (stored per component in order_risk_scores)

Each component value stored alongside composite score

Drives risk breakdown tooltip in Operations Dashboard — hover a risk badge to see which factors contribute most

Scores are append-only (always inserted, never updated) — produces a risk trajectory chart per order in the detail view

LLM explanation (claude-haiku-4-5-20251001)

Called after every score computation with the component breakdown as input

Produces a 1-sentence explanation stored with the score and shown in tooltips and alert bodies:

"Risk score 72: primarily driven by a 29% delay probability and 3 open BOM shortages on critical path components."

Where the score appears

Operations Dashboard — risk badge per order row in portfolio table

Order detail view — trajectory chart + component breakdown tooltip on hover

Alert bodies — score + 1-sentence explanation included in every at-risk alert

Design choice — Rule-based components (BOM, quality, logistics, anomaly) are fully auditable. The ML component adds forward-looking probability that rules alone cannot provide.

Natural-language queries over live lifecycle data

RAG using pgvector for semantic search and claude-opus-4-6 for generation. The assistant does not rely on the LLM's parametric knowledge for facts about orders — every answer is grounded in live DB data retrieved at query time.

Query routing (before retrieval or LLM call)

AGGREGATE

"How many orders are in QA holds right now?"

Answered from pre-computed DB query — no LLM or embedding needed

SPECIFIC

"What is blocking ORD-4821?"

Exact match by order ID — bypasses vector search, fetches that order's full context

SEMANTIC

"Which orders have supply chain risk this week?"

Full RAG flow — embed → pgvector search → top-5 retrieval → LLM generation

Full RAG query flow (semantic route)

- 1 Embed question via voyage-3 → 1024-dim query vector
- 2 pgvector cosine search WHERE org_id = :org_id LIMIT 5
- 3 Fetch live context for top-5 orders from PostgreSQL (not just the embedding snapshot)
- 4 Assemble RAG prompt → send to claude-opus-4-6
- 5 Response + cited order IDs returned → rendered as clickable links

Embedding generation (after each lifecycle DB write)

Worker generates text representation of updated order lifecycle state
Embedded via Anthropic voyage-3 → 1024-dimensional vector

Upserted into lifecycle_embeddings(order_id, embedding
vector(1024), text_snapshot, updated_at)

pgvector ivfflat index enables fast approximate nearest-neighbor search

Guardrails

Tenant isolation	pgvector query enforces WHERE org_id = :org_id — answers
No speculation	System prompt: 'Answer only from the provided context. If the answer is not in the context, say so. Never speculate.'
Read-only	The assistant has no write access to any data — retrieval and generation only
Data freshness	Responses include 'data as of' drawn from the most recent last_synced_at across retrieved records
Conversation	Last 6 turns included in prompt for follow-up support — stored in browser session, not server-side

Frontend rendering

Cited order IDs in responses are rendered as clickable links to the full order detail view in the Operations Dashboard.

Grounding strategy —

After retrieval, live context is fetched fresh from PostgreSQL — answers always reflect current order state

Deliver value in vertical slices — not horizontal layers

The guiding principle

Vertical slices, not horizontal layers

Each phase ships a complete, usable capability for at least one user group.
No phase ends with infrastructure that isn't yet visible to anyone.

The alternative — building the full data layer, then the full API layer, then the full frontend — delivers nothing of value until week 16.

The risk of that approach: requirements shift, priorities change, and the team spends months building something no one has tested with real users.

What we avoid

Week 1–4	Data model + infrastructure	✗ zero value
Week 5–8	All integrations	✗ zero value
Week 9–12	All API endpoints	✗ zero value
Week 13–16	All frontend views	✗ zero value
Week 17	First usable product delivered	✓ value

What we do instead

Phase 1 (Wk 1–6)

Clients see order status. Ops have a unified order list.

Value delivered at end of phase →

Phase 2 (Wk 7–10)

Manufacturing + quality visibility for ops. Executive KPIs.

Value delivered at end of phase →

Phase 3 (Wk 11–16)

Full lifecycle end-to-end. AI makes it proactive.

Value delivered at end of phase →

Phase 4 (Wk 17+)

Production-grade: real integrations, RAG, security hardening.

Value delivered at end of phase →

Principle — Every phase ends with working software in front of real users — not just a layer of infrastructure waiting for the next layer on top.

Four phases from foundation to production-grade platform

Each phase builds on the last. Source systems added incrementally — mock connectors from day one, swapped for real ones in Phase 4 without changing any core platform logic.

PHASE	W1	W4	W7	W10	W13	W16	W18	W20	VALUE DELIVERED
Phase 1 Foundation	Foundation								Clients log in, see order status. Ops teams have a unified order list.
Phase 2 Mfg & Quality	Mfg & Quality								Mfg + QA visibility. First executive KPI dashboard.
Phase 3 Intelligence					Intelligence				End-to-end lifecycle. AI makes it proactive, not reactive.
Phase 4 Production Ready								Production	Real integrations, RAG assistant, SOC 2 gap assessment complete.
CONNECTORS ACTIVE	Mock CRM + ERP			Mock MES + QMS		Mock TMS + FSM + ITSM			Live: CRM → ERP → MES → QMS → TMS → FSM → ITSM
Build strategy — Mock connectors generate deterministic synthetic data from day one. Real integrations added, prioritized by data quality and operational value.									

Foundation — Weeks 1–6

Goal: clients can log in and see their order status. Ops teams have a unified order list replacing manual spreadsheet tracking.

What gets built

Data & infrastructure

- Core lifecycle data model — full PostgreSQL schema
- ECS Fargate + RDS + Redis on AWS dev via Terraform
- AWS Cognito: 3 roles (client / ops / executive) + RBAC
- Mock CRM + ERP connector adapters (deterministic Faker data)
- SQS ingestion pipeline with Normalizer + Reconciler

Backend API

- GET /api/orders — order list with status and stage
- GET /api/orders/{id}/lifecycle — full timeline view
- GET /api/alerts — basic risk flags and alert feed
- JWT validation + org_id tenant isolation
- GitHub Actions CI/CD: test → build → ECR → ECS

Frontend

- Client Portal: 7-stage order timeline with milestone status

Weeks 1–6 — By the end of Phase 1, both the client portal and ops dashboard are deployed and working against mock CRM + ERP data on AWS.

- Cognito login flow + role-based routing

Value delivered at end of Phase 1

Client users

- ✓ Log in and see their order's current stage
- ✓ View milestone timeline with completion dates
- ✓ No need to contact account managers for status

Operations teams

- ✓ Single unified order list replacing spreadsheets
- ✓ Filter by customer, stage, or team
- ✓ Basic risk flags (High / Med / Low) per order

Leadership

- ✓ Platform deployed and accessible
- ✓ Authentication and multi-tenancy working
- ✓ Foundation ready for Phase 2 additions

Manufacturing & Quality — Weeks 7–10

Goal: operations teams see manufacturing and quality in a single view. Executives get a first version of the KPI dashboard.

What gets built

New connectors

- Mock MES adapter — work order progress, machine events
- Mock QMS adapter — inspection records, hold status, results
- SQS event-driven path for MES (near-real-time webhooks)

Operations Dashboard — additions

- Manufacturing progress panel: work order status, % complete
- Quality inspection panel: inspection records, hold status
- Reopen history and hold duration tracking per order

Executive Dashboard — first version

- Static KPI cards: OTD %, quality pass rate
- Order count tiles with trend direction
- Read replica routing for aggregate queries

Value delivered at end of Phase 2

Operations teams

- ✓ Manufacturing progress visible without accessing MES directly
- ✓ Quality holds surfaced in context — same view as order status
- ✓ BOM shortages and rework flags visible per order
- ✓ Single screen replaces cross-system status chasing

Executive leadership

- ✓ First live dashboard: OTD % and quality pass rate
- ✓ Order-at-risk count visible at portfolio level
- ✓ Foundation for Phase 3 trend charts and AI narrative

Weeks 7–10 — 4 weeks to add MES + QMS coverage and a first executive view. Adapter pattern means no existing code changes — only new adapters added.

Logistics, Installation & Intelligence — Weeks 11–16

Goal: full lifecycle visibility end-to-end. AI features make the platform proactive rather than just informational.

What gets built

New connectors

- Mock TMS adapter — shipment records, carrier tracking, ETAs
- Mock FSM adapter — installation records, site readiness
- Mock ITSM adapter — service tickets, SLA status

AI capabilities added

- Delay prediction model — trained on Phase 1–2 mock data
- Risk scoring engine — 5-component weighted score per order
- Lifecycle summaries — claude-opus-4-6, audience-specific prompts
- Anomaly detection — CUSUM + Z-score on MES / QMS metrics

Dashboard additions

- Client portal: logistics and installation stages + ETAs
- Ops: risk score badges, anomaly alerts, AI assistant panel
- Exec: trend charts, revenue-at-risk table, AI daily narrative

Value delivered at end of Phase 3

Clients

- ✓ 7-stage lifecycle visible end-to-end
- ✓ Confidence indicator on every delivery date
- ✓ AI-generated status summary in plain language

Operations

Risk scores per order · anomaly alerts · AI assistant active

Executives

Revenue-at-risk table · trend charts · daily AI narrative · full KPI suite

The AI inflection point

After Phase 3, the platform shifts from reactive reporting to proactive intelligence — risk is predicted before it becomes a problem, not discovered after.

Weeks 11–16 — Platform becomes genuinely differentiated. The data now drives predictions, anomaly detection, and AI-generated narratives.

Production Readiness — Weeks 17+

Goal: production-grade platform with real data, enterprise security, and the AI assistant — ready for rollout.

What gets built

Real system integrations (prioritized)
· 1. CRM + ERP — highest quality, most mature APIs
· 2. MES + QMS — highest operational value for mfg orgs
· 3. TMS — critical for client ETA accuracy
· 4. FSM + ITSM — post-delivery stages, lower urgency
AI Assistant (RAG)
· pgvector embeddings for all lifecycle records (voyage-3, 1024-dim)
· Natural-language Q&A: 'Which orders are most at risk this week?'
· Query routing: aggregate → DB, specific → exact match, semantic →
· 6-turn conversation history · browser-session storage only
Production hardening
· Performance: query tuning, cache strategy hardening
· Mobile-responsive design for client portal
· Security: pen test, SOC 2 gap assessment, audit logging

Phase 4 outcome — Production-grade platform with real integrations, AI assistant live, and SOC 2 gap assessment complete — ready for enterprise rollout.

Integration sequencing rationale

1	CRM + ERP Highest data quality · most mature REST APIs · foundational to all other stages
2	MES + QMS Highest operational impact for manufacturing orgs · enables AI model quality
3	TMS Critical for client ETA accuracy · logistics is the last customer-facing step
4	FSM + ITSM Post-delivery stages · lower urgency · added once core lifecycle is stable

Adapter pattern: each mock connector is replaced by its live counterpart independently. Zero changes to core pipeline, normalizer, or any downstream

Technology stack — deliberate choices for the problem at hand

Every technology in the stack was chosen for a specific reason — not defaults, not familiarity for its own sake. The table below shows what, and the right column explains why.

LAYER	TECHNOLOGY	WHY THIS CHOICE
Frontend	React 18+ · TypeScript · shadcn/ui · React Query · Vite	Type safety end-to-end. React Query handles server state, cache invalidation, and optimistic updates — critical for real-time data. Vite dev proxy mirrors CloudFront with zero config.
Backend API	Python 3.12 · FastAPI · SQLAlchemy (async) · Pydantic	Python is the natural choice when the same codebase runs ML inference and API serving. FastAPI's async support + Pydantic schemas give type-safe JSON with automatic validation.
Database	PostgreSQL 16 (RDS) · Redis (ElastiCache) · pgvector	PostgreSQL is the single source of truth for lifecycle records. pgvector enables RAG without a separate vector DB in Phase 3. Redis handles dashboard cache and WebSocket Pub/Sub.
Messaging	Amazon SQS FIFO · EventBridge Scheduler	FIFO guarantees ordered processing per order_id — critical for lifecycle event sequencing. EventBridge replaces cron for connector polling: configurable per-connector intervals.
AI / ML	XGBoost · numpy · Claude API · voyage-3 · pgvector	XGBoost for fast tabular inference at ingestion time. Claude (Opus for generation, Haiku for high-volume explanation). voyage-3 embeddings + pgvector = RAG without added infra.
Auth	AWS Cognito · OAuth 2.0 / OIDC · JWT (RS256)	Cognito handles SAML federation for enterprise SSO (Okta, Azure AD) out of the box. JWT claims carry org_id + role — tenant isolation is enforced at the ORM layer on every query.
Infrastructure	AWS ECS Fargate · RDS Multi-AZ · Terraform	Fargate eliminates server management; each service scales and deploys independently. Terraform ensures every environment is reproducible from scratch. No manual provisioning.
CI/CD	GitHub Actions → Amazon ECR → ECS rolling deploy	Per-service pipelines: backend and frontend deploy independently. Rolling deployment with ALB connection draining. terraform plan on every PR — apply requires manual approval.
Observability	AWS CloudWatch · structlog · X-Ray	Structured JSON logs carry trace_id, org_id, order_id — every request is traceable. Custom CloudWatch metrics (IngestLag, QueueDepth, CacheHitRate, AIResponseLatency) drive alarms.



From fragmented data to lifecycle intelligence

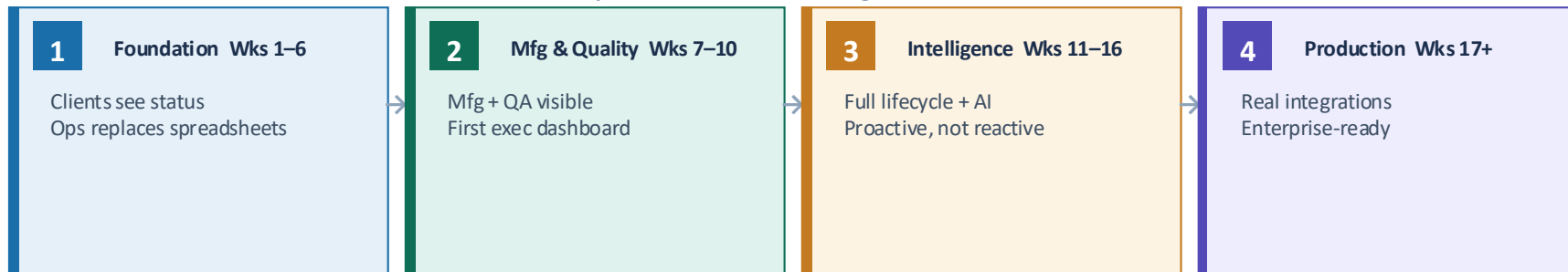
The problem CompassX solves

- ✗ 8 source systems, each with a partial view
- ✗ No single system knows the full lifecycle
- ✗ Status requires manual aggregation across teams
- ✗ Clients discover delays — no early warning
- ✗ Quality anomalies surface too late to act on
- ✗ No real-time view of delivery risk or revenue exposure

What the platform delivers

- ✓ Unified lifecycle view — single source of truth
- ✓ Clients see real-time status — no calls to ops
- ✓ Ops sees blockers and acts before delays occur
- ✓ Proactive alerts weeks before ETA breach
- ✓ Anomaly detection the moment issues emerge
- ✓ Executives see revenue exposure with AI context

Delivered in four phases — each with working software in front of real users



The platform reads from existing systems without replacing them — adding a layer of intelligence and visibility that no individual system can